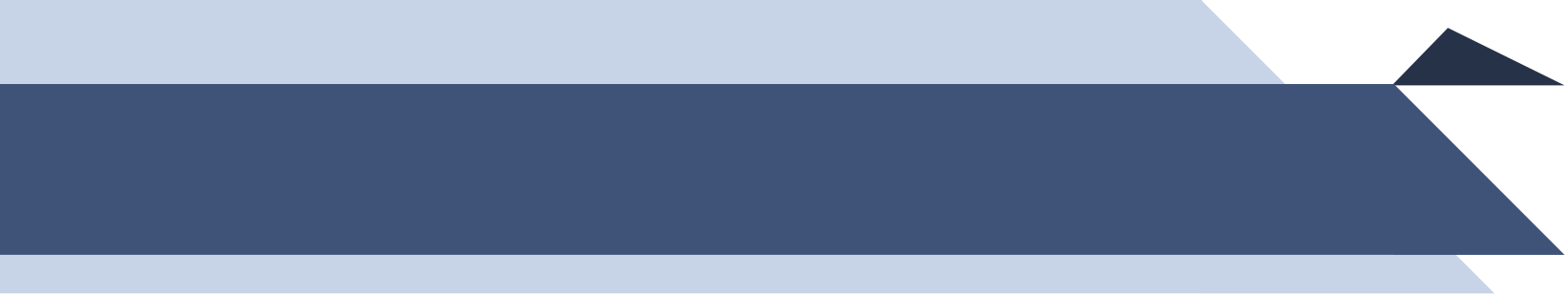


ENTERPRISE JAVA (21MCA202)

BY

Anantha Murthy
Assistant Professor
Dept. of MCA
NMAM.I.T, Nitte



Unit 3

2.Multithreaded Programming

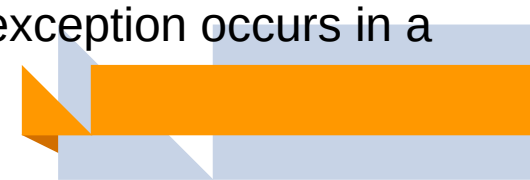




Multithreading

- Multithreading in Java is a process of executing multiple threads simultaneously.
- A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.
- However, we use multithreading than multiprocessing because threads use a shared memory area. They don't allocate separate memory area so saves memory, and context-switching between the threads takes less time than process.

Advantages of Java Multithreading

- 1) It doesn't block the user because threads are independent
 - 2) You can perform many operations together, so it saves time.
 - 3) Since Threads are independent, it doesn't affect other threads if an exception occurs in a single thread.
- 



Multitasking

Multitasking is a process of executing multiple tasks simultaneously. We use multitasking to utilize the CPU. Multitasking can be achieved in two ways:

Process-based Multitasking (Multiprocessing)

- Each process has an address in memory. In other words, each process allocates a separate memory area.
- A process is heavyweight.
- Cost of communication between the process is high.
- Switching from one process to another requires some time for saving and loading registers, memory maps, updating lists, etc.

Thread-based Multitasking (Multithreading)

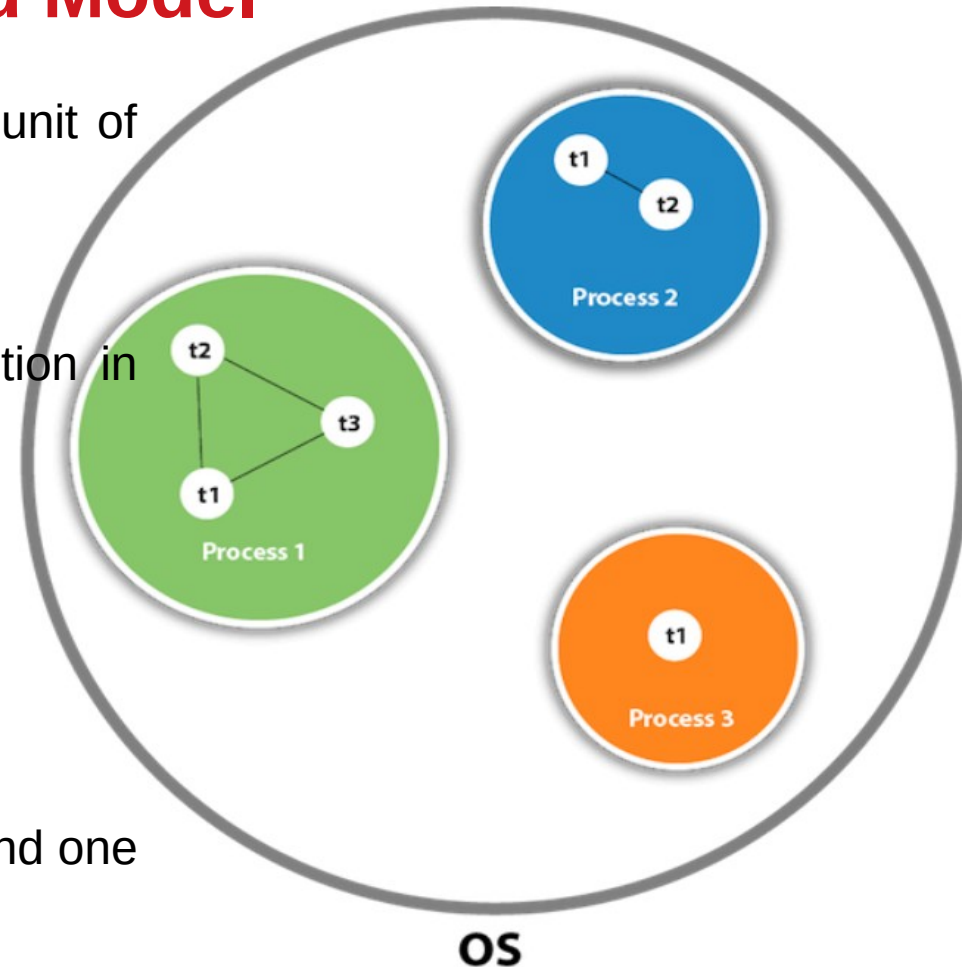
- Threads share the same address space.
- A thread is lightweight.
- Cost of communication between the thread is low.
- Inter-thread communication is less expensive compared to inter-process communication.



Java Thread Model

A thread is a lightweight subprocess, the smallest unit of processing. It is a separate path of execution.

- Threads are independent. If there occurs exception in one thread, it doesn't affect other threads.
- It uses a shared memory area.
- A thread is executed inside the process.
- There is context-switching between the threads.
- There can be multiple processes inside the OS, and one process can have multiple threads.

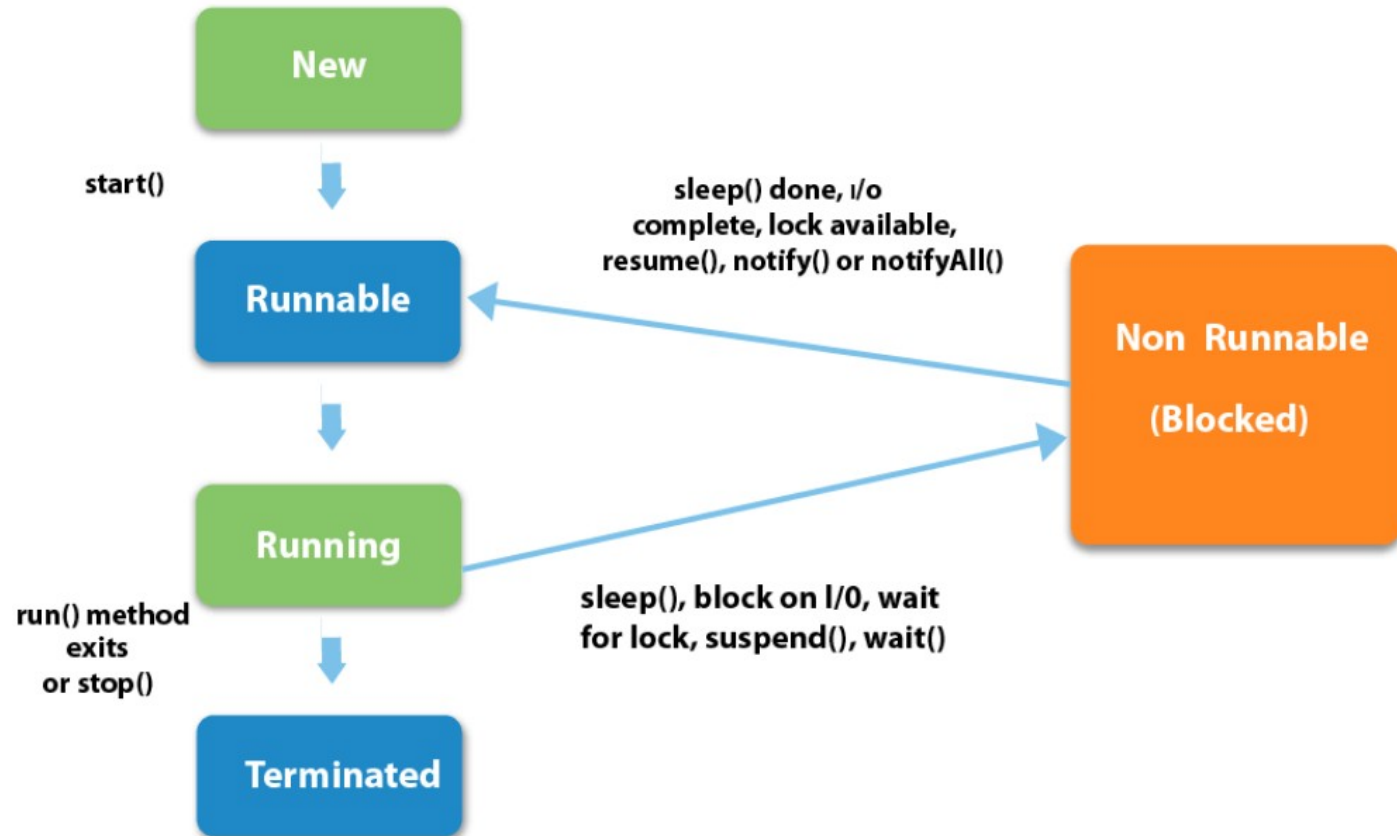


Life cycle of a Thread (Thread States)

The life cycle of the thread in java is controlled by JVM.

The java thread states are as follows:

1. New
2. Runnable
3. Running
4. Non-Runnable (Blocked)
5. Terminated





Life cycle of a Thread (Thread States)

1) New

The thread is in new state if you create an instance of Thread class but before the invocation of start() method.

2) Runnable

The thread is in runnable state after invocation of start() method, but the thread scheduler has not selected it to be the running thread.

3) Running

The thread is in running state if the thread scheduler has selected it.

4) Non-Runnable (Blocked)

This is the state when the thread is still alive, but is currently not eligible to run.

5) Terminated

A thread is in terminated or dead state when its run() method exits.





Main thread in Java

When a Java program starts up, one thread begins running immediately. This is usually called the **main thread** of program, because it is the one that is executed when program begins.

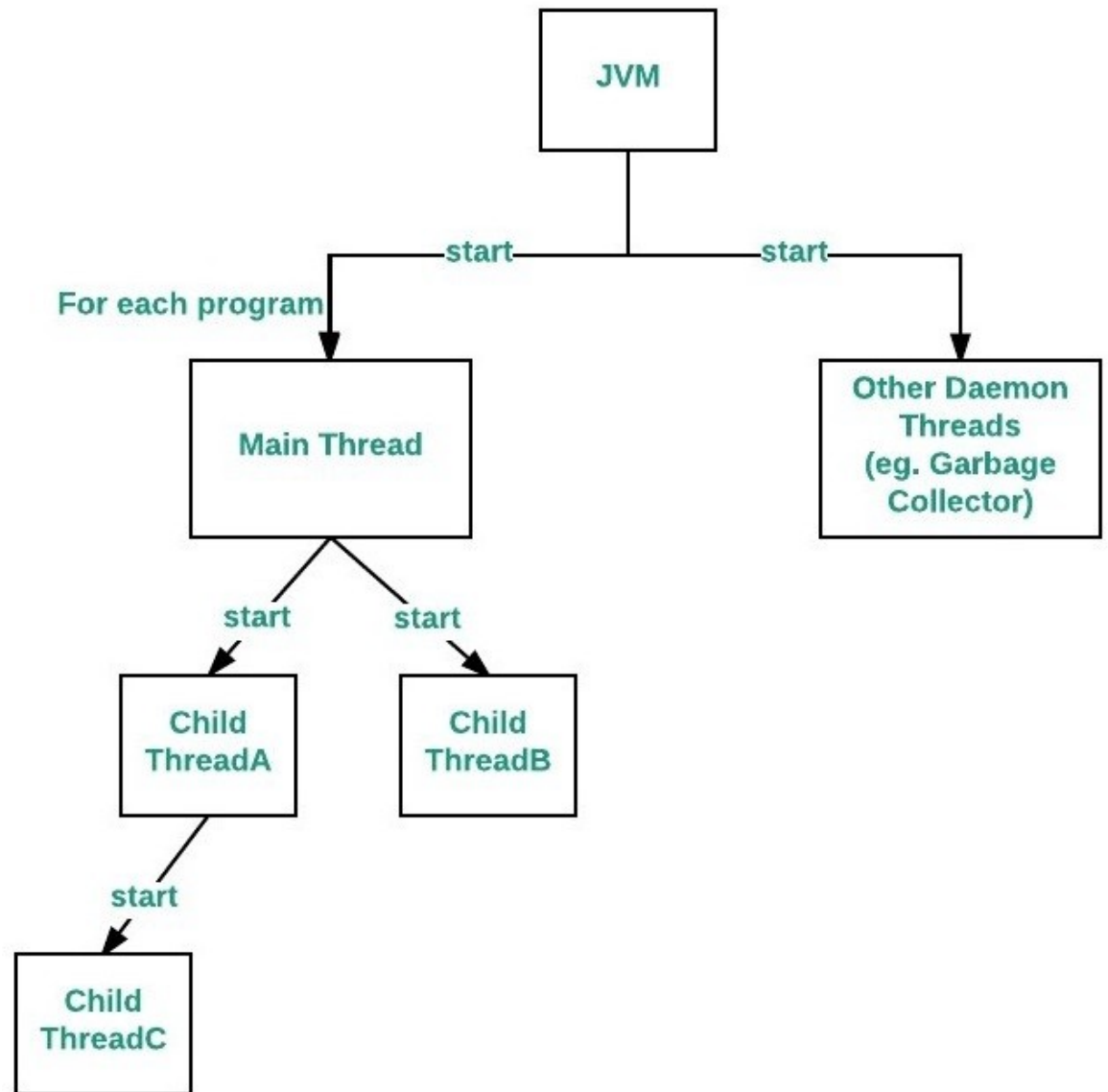
Properties :

- It is the thread from which other “**child**” **threads** will be spawned.
- Often, it must be the last thread to finish execution because it performs various shutdown actions

How to control Main thread

- To control main thread, we must obtain a **reference** to it.
 - This can be done by calling the method **currentThread()** which is present in **Thread class**.
 - This method returns a reference to the thread on which it is called.
- 

Main thread in Java



Main thread Example

```
class currentthread
{
    public static void main(String args[])
    {
        Thread t=Thread.currentThread();
        System.out.println("Current Thread:"+t);

        t.setName("My Thread");
        System.out.println("After changing name:"+t.getName());
        System.out.println("Its priority is:"+t.getPriority());

        try
        {
            for(int i=0;i<5;i++)
            {
                System.out.println(i);
                Thread.sleep(100);
            }
        }
        catch (InterruptedException e)
        {System.out.println("Main thread interrupted");}
    }
}
```

Output:

```
Current Thread:Thread[main,5,main]
After changing name:My Thread
Its priority is:5
0
1
2
3
4
```

Java Thread class

- Java provides Thread class to achieve thread programming.
- Thread class provides constructors and methods to create and perform operations on a thread.
- Thread class extends Object class and implements Runnable interface.

Java Thread Methods

S.N.	Modifier and Type	Method	Description
1)	void	<code>start()</code>	It is used to start the execution of the thread.
2)	void	<code>run()</code>	It is used to do an action for a thread.
3)	static void	<code>sleep()</code>	It sleeps a thread for the specified amount of time.



Java Thread class Methods

4)	static Thread	<code>currentThread()</code>	It returns a reference to the currently executing thread object.
5)	void	<code>join()</code>	It waits for a thread to die.
6)	int	<code>getPriority()</code>	It returns the priority of the thread.
7)	void	<code>setPriority()</code>	It changes the priority of the thread.
8)	String	<code>getName()</code>	It returns the name of the thread.
9)	void	<code>setName()</code>	It changes the name of the thread.
10)	long	<code>getId()</code>	It returns the id of the thread.
11)	boolean	<code>isAlive()</code>	It tests if the thread is alive.



Java Thread class Methods

12)	static void	<code>yield()</code>	It causes the currently executing thread object to pause and allow other threads to execute temporarily.
13)	void	<code>suspend()</code>	It is used to suspend the thread.
14)	void	<code>resume()</code>	It is used to resume the suspended thread.
15)	void	<code>stop()</code>	It is used to stop the thread.
16)	void	<code>destroy()</code>	It is used to destroy the thread group and all of its subgroups.
17)	void	<code>notify()</code>	It is used to give the notification for only one thread which is waiting for a particular object.
18)	void	<code>notifyAll()</code>	It is used to give the notification to all waiting threads of a particular object.

How to create thread

There are two ways to create a thread:

- By implementing Runnable interface.
- By extending Thread class.

Thread class:

Thread class provide constructors and methods to create and perform operations on a thread.

Thread class **extends** **Object** class and **implements** **Runnable** interface.

Commonly used Constructors of Thread class:

Thread()

Thread(String name)

Thread(Runnable r)

Thread(Runnable r,String name)

Implementing the Runnable Interface

The easiest way to create a thread is to create a class that implements the runnable interface. After implementing runnable interface, the class needs to implement the **run()** method.

Run Method Syntax: ***public void run()***

It introduces a concurrent thread into your program. This thread will end when run() method terminates.

- You must specify the code that your thread will execute inside run() method.
- run() method can call other methods, can use other classes and declare variables just like other normal method.

Implementing the Runnable Interface

The **Runnable interface** should be implemented by any class whose instances are intended to be executed by a thread. Runnable interface have only one method named `run()` and is used to perform action for a thread.

Starting a thread:

start() method of Thread class is used to start a newly created thread. It performs following tasks:

- A new thread starts(with new callstack).
- The thread moves from New state to the Runnable state.
- When the thread gets a chance to execute, its target `run()` method will run.

Implementing the Runnable Interface

Steps :

1. Declare the class as implementing Runnable interface.
2. Implement the `run()` method.
3. Create a thread by defining an object that is instantiated from the runnable interface implemented class as the target of the thread.
4. Call the thread's `start()` method to run the thread.

Example:

```
Mythread m=new Mythread();
```

```
Thread T=new Thread(m);
```

```
T.start();
```

Or

```
new Thread(new Mythread()).start();
```

Implementing the Runnable Interface

Note:

If you are implementing Runnable interface in your class, then you need to explicitly create a Thread class object and need to pass the Runnable interface implemented class object as a parameter in its constructor.

```
class MyThread implements Runnable
{
    public void run()
    {
        System.out.println("concurrent thread started running..");
    }
}

class MyThreadDemo
{
    public static void main(String args[])
    {
        MyThread mt = new MyThread();
        Thread t = new Thread(mt);
        t.start();
    }
}
```

Output:

```
concurrent thread started running..
```



Extending Thread class

This is another way to create a thread by a new class that extends Thread class and create an instance of that class.

The extending class must override run() method which is the entry point of new thread.

Steps :

1. Declare the class as extending the Thread class.
2. Implement the **run()** method.
3. Create a thread object.
4. Call the thread's **start()** method to initiate the thread execution.



Extending Thread class

In this case also, we must override the run() and then use the start() method to run the thread.

Also, when you create MyThread class object, Thread class constructor will also be invoked, as it is the super class, hence MyThread class object acts as Thread class object.

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("concurrent thread started running..");
    }
}

class MyThreadDemo
{
    public static void main(String args[])
    {
        MyThread mt = new MyThread();
        mt.start();
    }
}
```

Output:

```
concurrent thread started running..
```



Extending Thread class

Some Important points to Remember

- When we extend Thread class, we cannot override ***setName()*** and ***getName()*** functions, because they are declared final in Thread class.
- While using sleep(), always handle the exception it throws.

static void sleep(long milliseconds) throws InterruptedException

Questions :

1. What if we call run() method directly without using start() method?
2. Can we Start a thread twice?



isAlive() and join() methods of Thread Class

Q) How can one thread know when another thread has ended?

A) Java multi-threading provides two ways to find that

1) isAlive()

Syntax :

final boolean isAlive()

Return Value:

Returns true if the thread upon which it is called is still running.

It returns false otherwise.

2) join()

Syntax :

final void join() throws InterruptedException

isAlive() and join() methods of Thread Class

isAlive() :

- It tests if this thread is alive.
- A thread is alive if it has been started and has not yet died.
- There is a transitional period from when a thread is running to when a thread is not running. After the run() method returns, there is a short period of time before the thread stops.
- If we want to know if the start method of the thread has been called or if thread has been terminated, we must use isAlive() method.
- This method is used to find out if a thread has actually been started and has yet not terminated.

isAlive() and join() methods of Thread Class

join() :

- When the join() method is called, the current thread will simply wait until the thread it is joining with is no longer alive.
- Or we can say the method that you will more commonly use to wait for a thread to finish is called join().
- This method waits until the thread on which it is called terminates.
- Its name comes from the concept of the calling thread waiting until the specified thread joins it.
- Additional forms of join() allow you to specify a maximum amount of time that you want to wait for the specified thread to terminate.

Syntax: *final void join(long milliseconds) throws InterruptedException*

isAlive() and join() methods of Thread Class

```
class joinandalive1 extends Thread
{
    public void run()
    {
        System.out.println("Hi thread1..");

        try
        {
            Thread.sleep(500);
        }catch (InterruptedException e){}
        System.out.println("Exiting thread1...");
    }
}
```

```
class joinandalive2 extends Thread
{
    public void run()
    {
        System.out.println("Hi thread2..");

        try
        {
            Thread.sleep(1000);
        }catch (InterruptedException e){}
        System.out.println("Exiting thread2...");
    }
}
```

```
class joinandalive
{
    public static void main(String args[])
    {
        joinandalive1 j1=new joinandalive1();
        joinandalive2 j2=new joinandalive2();
        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        j1.start();
        j2.start();

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());

        /*
        //This is the first method to make main thread to finish its execution last
        try{Thread.sleep(1000);}catch(InterruptedException e){}*/

        // Here main may exit before thread2 but waits for thread1 to finish
        try
        {
            j1.join();
        }catch (InterruptedException e){}

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        System.out.println("Main thread exiting..");
    }
}
```

isAlive() and join() methods of Thread Class

Output:

```
false
false
true
true
Hi thread1..
Hi thread2..
Exiting thread1...
false
true
Main thread exiting..
Exiting thread2...
```

```
class joinandalive
{
    public static void main(String args[])
    {
        joinandalive1 j1=new joinandalive1();
        joinandalive2 j2=new joinandalive2();
        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        j1.start();
        j2.start();

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());

        //Here main thread waits for both thread1 and thread2 to finish
        try
        {
            j1.join();
            j2.join();
        }catch (InterruptedException e){}

        System.out.println(j1.isAlive());
        System.out.println(j2.isAlive());
        System.out.println("Main thread exiting..");
    }
}
```

Output:

```
false
false
true
true
Hi thread1..
Hi thread2..
Exiting thread1...
Exiting thread2...
false
false
Main thread exiting..
```



Synchronization

Synchronization in java is the capability to control the access of multiple threads to any shared resource.

Java Synchronization is better option where we want to allow only one thread to access the shared resource.

The synchronization is mainly used to

- To prevent thread interference.
- To prevent consistency problem.





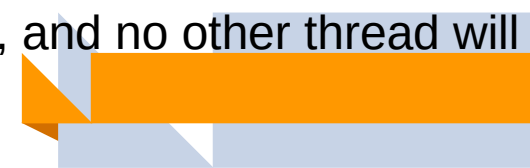
Synchronization

Why we need Synchronization?

If we do not use synchronization, and let two or more threads access a shared resource at the same time, it will lead to distorted results.

Consider an example, Suppose we have two different threads T1 and T2, T1 starts execution and **save certain values in a file temporary.txt** which will be used to calculate some result when T1 returns. Meanwhile, T2 starts and before T1 returns, **T2 change the values saved by T1 in the file temporary.txt** (temporary.txt is the shared resource). Now obviously T1 will return wrong result.

To prevent such problems, synchronization was introduced. With synchronization in above case, once T1 starts using temporary.txt file, this file will be locked(LOCK mode), and no other thread will be able to access or modify it until T1 returns.





Types of Thread Synchronization

There are two types of thread synchronization.

1. Mutual Exclusive

Mutual Exclusive helps keep threads from interfering with one another while sharing data.

a. Synchronized method.

b. Synchronized block.

2. Cooperation (Inter-thread communication)

It is all about allowing synchronized threads to communicate with each other.



Understanding the problem without Synchronization

```
class First
{
    public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}
```

```
class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        fobj.display(msg);
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second(fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[new[welcome[programmer]
]
```



Java synchronized method

- If you declare any method as synchronized, it is known as synchronized method.
- Synchronized method is used to lock an object for any shared resource.
- When a thread invokes a synchronized method, it automatically acquires the lock for that object and releases it when the thread completes its task.

- **Syntax :**

synchronized method_header

{

// Block of code

}



Understanding the problem with Synchronized method

```
class First
{
    synchronized public void display(String msg)
    {
        System.out.print ("["+msg);
        try
        {
            Thread.sleep(1000);
        }
        catch (InterruptedException e)
        {
            e.printStackTrace();
        }
        System.out.println ("]");
    }
}
```

```
class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        fobj.display(msg);
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second(fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[welcome]
[programmer]
[new]
```

Using Synchronized block

- If want to synchronize access to an object of a class or only a part of a method to be synchronized then we can use synchronized block for it.
- It is capable to make any part of the object and method synchronized.
- Suppose you have 50 lines of code in your method, but you want to synchronize only 5 lines, you can use synchronized block.
- If you put all the codes of the method in the synchronized block, it will work same as the synchronized method.

- **Syntax:**

synchronized (object)

{

// statements to be synchronized

}

Using Synchronized block

```
class First
{ public void display(String msg)
{
    System.out.print ("["+msg);
    try
    {
        Thread.sleep(1000);
    }
    catch (InterruptedException e)
    {
        e.printStackTrace();
    }
    System.out.println ("]");
}
}
```

```
class Second extends Thread
{
    String msg;
    First fobj;
    Second (First fp,String str)
    {
        fobj = fp;
        msg = str;
        start();
    }
    public void run()
    {
        synchronized(fobj)    //Synchronized block
        {
            fobj.display(msg);
        }
    }
}
```

```
public class Main
{
    public static void main (String[] args)
    {
        First fnew = new First();
        Second ss = new Second(fnew, "welcome");
        Second ss1= new Second (fnew,"new");
        Second ss2 = new Second(fnew, "programmer");
    }
}
```

Output:

```
[welcome]
[programmer]
[new]
```

Difference between synchronized keyword and synchronized block

When we use synchronized keyword with a method, it acquires a lock in the object for the whole method. It means that no other thread can use any synchronized method until the current thread, which has invoked its synchronized method, has finished its execution.

synchronized block acquires a lock in the object only between parentheses after the synchronized keyword. This means that no other thread can acquire a lock on the locked object until the synchronized block exits. But other threads can access the rest of the code of the method.

Q) Which is more preferred - Synchronized method or Synchronized block?



Inter-thread Communication

- Inter-thread communication or **Co-operation** is all about allowing synchronized threads to communicate with each other.
- Cooperation (Inter-thread communication) is a mechanism in which a thread is paused running in its **critical section** and another thread is allowed to enter (or lock) in the same critical section to be executed.
- A critical section is a block of code that accesses a **shared resource** and **can't be executed by more than one thread** at the same time.





Inter-thread Communication

➤ It is implemented by following methods of Object class:

wait() : tells calling thread to give up monitor and go to sleep until some other thread enters the same monitor and call notify.

notify(): wakes up a thread that called wait() on same object.

notifyAll(): wakes up all the thread that called wait() on same object.

All the three method can be called only from within a synchronized context





Inter-thread Communication

Syntaxes :

wait():

public final void wait()throws InterruptedException

public final void wait(long timeout) throws InterruptedException

notify():

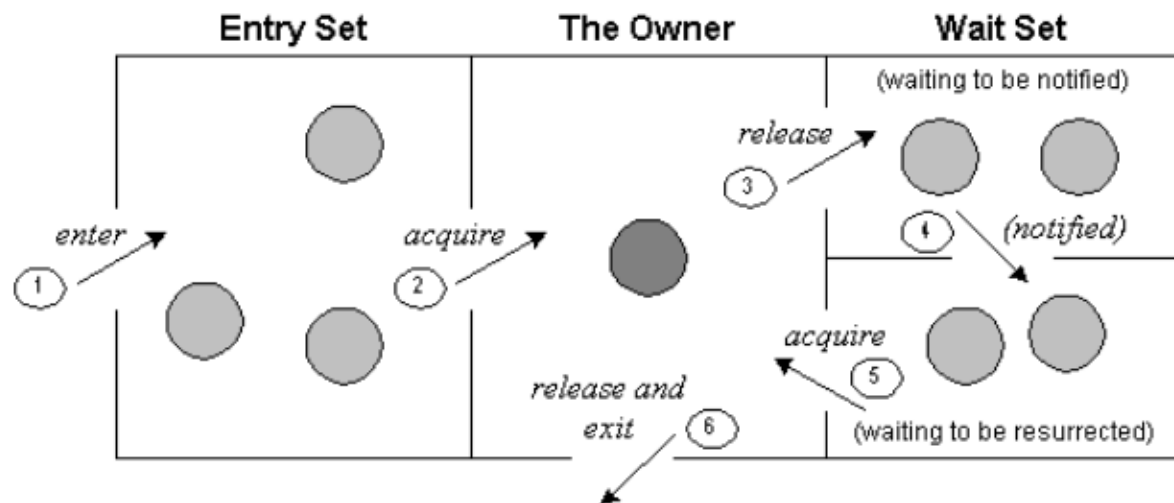
public final void notify()

notifyAll():

public final void notifyAll()



Understanding the process of inter-thread communication



The point to point explanation of the above diagram is as follows:

1. Threads enter to acquire lock.
2. Lock is acquired by one thread.
3. Now thread goes to waiting state if you call `wait()` method on the object. Otherwise it releases the lock and exits.
4. If you call `notify()` or `notifyAll()` method, thread moves to the notified state (runnable state).
5. Now thread is available to acquire lock.
6. After completion of the task, thread releases the lock and exits the monitor state of the object.

Inter-thread Communication

```
class Customer
{
    int amount=10000;

    synchronized void withdraw(int amount){
        System.out.println("going to withdraw...");

        if(this.amount<amount){
            System.out.println("Less balance; waiting for deposit...");
            try
            {wait();}
            catch(Exception e){}
        }
        this.amount-=amount;
        System.out.println("withdraw completed...");
    }

    synchronized void deposit(int amount){
        System.out.println("going to deposit...");
        this.amount+=amount;
        System.out.println("deposit completed... ");
        notify();
    }
}
```

```
class Test{
    public static void main(String args[]){
        final Customer c=new Customer();
        new Thread(){
            public void run(){c.withdraw(15000);}
        }.start();
        new Thread(){
            public void run(){c.deposit(10000);}
        }.start();
    }
}
```

Output:

```
going to withdraw...
Less balance; waiting for deposit...
going to deposit...
deposit completed...
withdraw completed...
```

Differences between wait() and sleep()

wait()	sleep()
wait() method releases the lock	sleep() method doesn't release the lock.
is the method of Object class	is the method of Thread class
is the non-static method	is the static method
is the non-static method	is the static method
should be notified by notify() or notifyAll() methods	after the specified amount of time, sleep is completed.
called from synchronised block	no such requirement
wait() is generally used on condition	sleep() method is simply used to put your thread on sleep.